

CSCE 411: Mastery Quiz 4

Name: _____

UIN: _____

Give an algorithm which takes as input a sorted array A of n integers (which are *not* guaranteed to be distinct) and returns the number of zeros in A . For example, given the array $[-5, -5, -1, 0, 0, 0, 1, 1, 2, 2, 2, 2, 5, 8, 10, 10, 10, 10]$, your algorithm should output 3.

- You may assume that A contains at least one 0
- Your algorithm should run faster than $\Theta(n)$ time.
- You should briefly justify the correctness of your algorithm (no need for a full induction proof).
- You should carefully analyze the worst-case asymptotic runtime of your algorithm (don't just state a big- O bound, justify why your algorithm satisfies that bound).
- If you want to modify an algorithm from the course notes or lecture slides, you should make sure to carefully explain how you are modifying the algorithm.

Solution: We give the following algorithm

```
NUMZEROS( $A[1, \dots, n]$ )
 $u \leftarrow n$ 
 $\ell \leftarrow 1$ 
While  $u \neq \ell$ :
     $\text{mid} \leftarrow \lfloor (u + \ell) / 2 \rfloor$ 
    If  $A[\text{mid}] \geq 0$ :
         $u \leftarrow \text{mid}$ 
    Else:
         $\ell \leftarrow \text{mid} + 1$ 
first_zero  $\leftarrow \ell$ 
 $u \leftarrow n + 1$ 
While  $u \neq \ell$ :
     $\text{mid} \leftarrow \lfloor (u + \ell) / 2 \rfloor$ 
    If  $A[\text{mid}] = 0$ :
         $\ell \leftarrow \text{mid} + 1$ 
    Else:
         $u \leftarrow \text{mid}$ 
first_positive  $\leftarrow \ell$ 
Return first_positive - first_zero
```

Correctness: We note that our algorithm binary searches for the first zero (whose index is greater than the index of any negative element but less than or equal to the index of any zero), then binary searches for the first positive element (whose index is greater than the index of any 0 and less than or equal to the index of any positive element). The difference between these two indices is the number of zeros.

Runtime: We note that each while loop runs until the size of the interval being searched ($u - \ell + 1$) becomes 1. Since the size of this interval is at most n to begin with and it halves each iteration of the loop, each while loop runs for $O(\log n)$ iterations. Since u, ℓ , and mid all store values which are at most n , they fit in integer RAM model variables and all of the arithmetic inside of the loop takes constant time. Therefore each loop takes $O(\log n)$ time. The operations outside of the loops also take constant time, so the overall runtime of our algorithm is $O(\log n)$.